

nginx JS — Dynamic Reconfiguration Guide (for single worker)

Table of Contents

1. Overview	2
1.1. Terminology	2
2. write_mode and read_mode	3
2.1. Default behaviour	3
2.2. write_mode values	3
2.3. read_mode values	3
2.4. API	4
2.5. Flow diagram: write_mode interactions	4
2.6. Example: per-tenant root directory	4
2.7. Example: persistent global mutation (hot reload)	5
3. suspendAcceptance() and resumeAcceptance()	5
3.1. Purpose	5
3.2. Example: atomic batch config update	6
3.3. Example: atomic batch with rollback	6
3.4. Flow diagram	7
4. snapshot() and restore()	7
4.1. What snapshot() captures	7
4.2. What restore() does	8
4.3. restore() mode override	9
4.4. Interaction table: snapshot × write_mode	9
4.5. Flow diagram: snapshot lifecycle	9
4.6. Flow diagram: local isolation with deliberate commit	10
5. Complete Examples	11
5.1. Example 1: Rollback on Error	11
5.2. Example 2: Atomic Two-Phase Apply	11
5.3. Example 3: Upstream Peer Weight Balancing with Rollback	12
5.4. Example 4: Single-Peer Snapshot for Fine-Grained Rollback	12
5.5. Example 5: A/B Configuration Experiment	13
5.6. Example 6: Config Snapshot for Audit	14
5.7. Example 7: Guarded Global Mutation	14
6. API Reference	15
6.1. location.setWriteMode(mode)	15

6.2. <code>location.setReadMode(mode)</code>	15
6.3. <code>location.snapshot()</code>	15
6.4. <code>upstream.snapshot()</code>	15
6.5. <code>peer.snapshot()</code> / <code>rrPeer.snapshot()</code>	16
6.6. <code>snap.restore([options])</code>	16
6.7. <code>nginx.suspendAcceptance()</code>	16
6.8. <code>nginx.resumeAcceptance()</code>	17
6.9. <code>NginxSnapshot</code> (opaque object)	17
7. Design Notes	17
7.1. Internal snapshot vs JS snapshot	17
7.2. Why <code>restore()</code> is best-effort	17
7.3. Snapshot scope for <code>upstream.snapshot()</code>	18



DRAFT FOR COMMENTS — This document is a work in progress. Content, API names, and examples are subject to change before release.

1. Overview

The nginx JS DOM lets you read **and write** live nginx configuration from JavaScript running inside nginx workers. This guide covers the **write-side** of that API:

- `write_mode` / `read_mode` — control **where** property assignments go and where reads come from (global shared struct vs per-request snapshot).
- `snapshot()` / `restore()` — capture a point-in-time copy of a COM sub-tree and roll it back at any moment.

Together these two mechanisms let you implement safe hot-reload, per-tenant routing, A/B configuration experiments, and rollback-on-error patterns without reloading nginx.

1.1. Terminology

Term	Meaning
global struct	The C struct shared by all workers that was set up at config parse time. Mutations here affect every new request immediately.
per-request snapshot	A shallow copy of <code>r->loc_conf[]</code> (pointer array) plus deep copies of whatever individual C structs have been mutated for this request. Exists only for the duration of one request.
COM node	A JavaScript wrapper object (<code>NginxLocation</code> , <code>NginxProxy</code> , <code>NginxPeer</code> , <code>NginxRRPeer</code> , <code>NginxUpstream</code> , ...) backed by a nginx C struct.

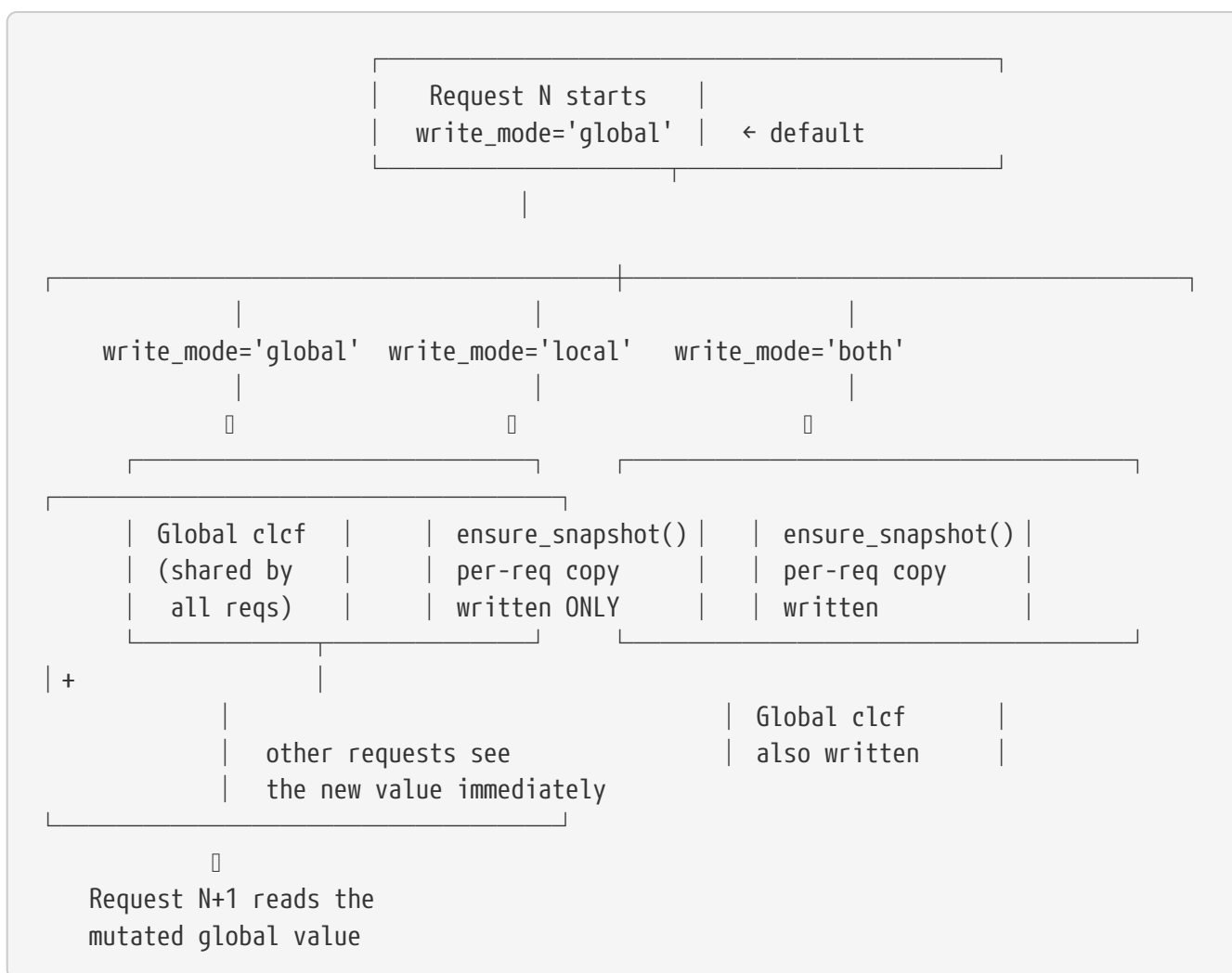
Value	Meaning
'local'	Property reads prefer the per-request snapshot when one exists, falling back to the global struct automatically (because <code>r->loc_conf[idx] == op->clcf</code> until a snapshot is taken).

2.4. API

```
req.location.setWriteMode(mode)  // 'global' | 'local' | 'both'
req.location.setReadMode(mode)   // 'global' | 'local'
```

Both methods update the opaque state of the `NginxLocation` wrapper **and** propagate the new `write_mode` into the per-request context (`rctx`) so that sub-object setters (`proxy`, `gzip`, `headers`, `rewrite`) also honour it.

2.5. Flow diagram: write_mode interactions



2.6. Example: per-tenant root directory

```
loc.handler = (req) => {
```

```
// Isolate writes to this request – global is never touched
req.location.setWriteMode('local');
req.location.setReadMode('local');

const tenant = req.getVar('http_x_tenant') || 'default';
req.location.root = '/var/www/' + tenant;

// req.sendfile still works with the local root
req.sendfile('/index.html');
};
```

Other concurrent requests continue to read the original global `root` because this request only wrote to its per-request snapshot.

2.7. Example: persistent global mutation (hot reload)

```
// Endpoint that updates the upstream peer weight globally
nginx.http.servers.forEach(srv => {
  const loc = srv.locations.find(l => l.path === '/admin/set-weight');
  if (!loc) { return; }

  loc.handler = (req) => {
    // Default write_mode='global' – affects all future requests
    const body = JSON.parse(req.body);
    const ups = nginx.http.upstreams.find(u => u.name === body.upstream);
    if (!ups) { req.respond(404, {}, 'upstream not found'); return; }

    const peer = ups.peers.find(p => p.address === body.peer);
    if (!peer) { req.respond(404, {}, 'peer not found'); return; }

    peer.weight = body.weight;
    req.json({ applied: true });
  };
});
```

3. suspendAcceptance() and resumeAcceptance()

3.1. Purpose

JavaScript within a worker is single-threaded, but nginx is event-driven: the event loop may dispatch a second request between any two consecutive assignments in a handler. Without suspension a concurrent request on this worker could observe a *half-updated* config:

```
req.location.root = '/new'; // ← request B starts here ...
```

```
req.location.proxy.readTimeout = 5000;    //    ... and misses this write
```

`nginx.suspendAcceptance()` removes all listening socket read events from this worker's event loop. No new requests are dispatched by this worker while acceptance is suspended; incoming TCP connections queue harmlessly in the OS socket backlog (up to `net.core.somaxconn`). Other workers continue accepting normally.

Call `nginx.resumeAcceptance()` once the batch is complete.



Both functions are **synchronous** and return `undefined`. They are only callable from a worker process — not from config phase, and not from a Worker or SharedWorker thread.

For changes that must be consistent across **all** workers, use `nginx.suspendAllWorkers()` / `nginx.resumeAllWorkers()` instead — see the *for all workers* guide.

3.2. Example: atomic batch config update

```
loc.handler = async (req) => {
  const cfg = JSON.parse(await req.readBody());

  nginx.suspendAcceptance();           // no new requests on this
  worker

  req.location.root                    = cfg.root;
  req.location.gzip.level              = cfg.gzip.level;
  req.location.proxy.readTimeout      = cfg.proxy.readTimeout;
  req.location.proxy.connectTimeout  = cfg.proxy.connectTimeout;

  nginx.resumeAcceptance();           // restore normal accept
  behaviour

  req.json({ applied: true });
};
```

3.3. Example: atomic batch with rollback

Always call `resumeAcceptance()` even when an exception is thrown. The `try/finally` pattern guarantees this:

```
loc.handler = async (req) => {
  const snap = req.location.snapshot();    // save rollback point

  nginx.suspendAcceptance();

  try {
```

```

const cfg = JSON.parse(await req.readBody());

req.location.root          = cfg.root;
req.location.proxy.readTimeout = cfg.proxy.readTimeout;
req.location.gzip.level     = cfg.gzip.level;

req.json({ applied: true });
} catch (e) {
  snap.restore();           // revert partial mutations
  req.respond(500, {}, e.message);
} finally {
  ngx.resumeAcceptance();   // always re-enable
}
};

```



Do **not** perform async I/O (network probes, `ngx.setTimeout`) inside the suspension window. Awaiting I/O while suspended keeps this worker stopped — it cannot drain its timer queue or respond to existing keep-alive requests. Validate config **before** calling `suspendAcceptance()`, then execute only synchronous writes inside the window.

3.4. Flow diagram

Request handler:

Event loop (this worker):

```

ngx.suspendAcceptance()
|  removes listen read events

```

```

|
|
|

```

```

req.location.root = '/a'
req.location.gzip.level = 3
req.location.proxy.readTimeout = 5000

```

```

ngx.resumeAcceptance()
|  re-adds listen read events

```

no new connections accepted
existing requests may complete

(no interleaving possible here)

queued connections accepted

4. snapshot() and restore()

4.1. What snapshot() captures

`obj.snapshot()` reads the **current visible values** of all settable properties on `obj` (through `read_mode`) and stores them in a `NginxSnapshot` handle. For `NginxLocation` it also recursively captures

the settable fields of sub-objects (`proxy`, `gzip`, `headers`, `rewrite`).

COM class	Fields captured by snapshot()
<code>NginxLocation</code>	<code>root</code> , <code>handler</code> , <code>sendfile</code> , <code>tcpNopush</code> , <code>tcpNodelay</code> , <code>etag</code> , <code>keepaliveTimeout</code> , <code>keepaliveRequests</code> , <code>clientMaxBodySize</code> , <code>clientBodyTimeout</code> , <code>sendTimeout</code> , <code>defaultType</code> , <code>alias</code> , <code>satisfy</code> , <code>limitExcept</code> , <code>lingering</code> , <code>lingeringTimeout</code> , <code>lingeringTime</code> , <code>resolverTimeout</code> , <code>chunkedTransferEncoding</code> , <code>msieRefresh</code> , <code>logNotFound</code> , <code>logSubrequest</code> , <code>recursiveErrorPages</code> , <code>clientBodyBufferSize</code> , <code>clientBodyInFileOnly</code> , <code>clientBodyInSingleBuffer</code> , <code>resetTimedoutConnection</code> , <code>absoluteRedirect</code> , <code>serverNameInRedirect</code> , <code>portInRedirect</code> , <code>msiePadding</code> , <code>ifModifiedSince</code> , <code>maxRanges</code> , <code>authDelay</code> , <code>keepaliveTime</code> , <code>sendLowat</code> , <code>postponeOutput</code> , <code>keepaliveDisable</code> , <code>keepaliveMinTimeout</code> , <code>sendfileMaxChunk</code> , <code>readAhead</code> , <code>directio</code> , <code>directioAlignment</code> , <code>errorPage</code>
<code>NginxLocation</code> (sub: <code>proxy</code>)	<code>httpVersion</code> , <code>connectTimeout</code> , <code>sendTimeout</code> , <code>readTimeout</code> , <code>buffering</code> , <code>requestBuffering</code> , <code>interceptErrors</code> , <code>bufferSize</code> , <code>nextUpstreamTries</code> , <code>nextUpstreamTimeout</code>
<code>NginxLocation</code> (sub: <code>gzip</code>)	<code>enable</code> , <code>level</code> , <code>minLength</code> , <code>vary</code>
<code>NginxLocation</code> (sub: <code>headers</code>)	<code>headersInherit</code> , <code>trailersInherit</code>
<code>NginxLocation</code> (sub: <code>rewrite</code>)	<code>log</code> , <code>uninitializedVariableWarn</code> , <code>stackSize</code>
<code>NginxPeer</code> / <code>NginxRRPeer</code>	<code>weight</code> , <code>maxFails</code> , <code>down</code> , <code>failTimeout</code> , <code>maxConns</code>
<code>NginxUpstream</code>	All current peers (one sub-node per peer, each with the fields above)

Null/undefined getter results are silently skipped (e.g. `alias` on a root-based location returns null and is not stored).

4.2. What `restore()` does

`snap.restore()` iterates all stored nodes and calls the standard property **setter** for each captured value using `JS_SetPropertyStr`.

The setter path handles:

- `write_mode` routing (global / local / both).
- `ngx_js_ensure_core_snapshot()` / `ngx_js_ensure_module_snapshot()` — creating per-request snapshot copies when needed.
- Lock acquisition for `NginxRRPeer` properties.

`restore()` is **best-effort**: individual setter errors are silently cleared so that one failed field does not prevent others from being restored.

`restore()` is **idempotent**: you can call it multiple times safely.

4.3. restore() mode override

```

snap.restore()                // use current write_mode (default)
snap.restore({ mode: 'global' }) // force all writes to global struct
snap.restore({ mode: 'local' }) // force all writes to per-req snapshot
snap.restore({ mode: 'both' })  // write to both

```

The override temporarily sets `loc_op→write_mode` and `rctx→write_mode` for the duration of the restore call, then restores the original values.

4.4. Interaction table: snapshot × write_mode

Scenario	snapshot captures	restore() writes to
write_mode='global' default, no override	current global values	global (other reqs see it)
write_mode='local' no override	current local (= global if no prior mutation)	local snapshot only (global untouched)
write_mode='both' no override	current local (mirrors global)	both global and local snapshot
Any write_mode + restore({mode:'global'})	whatever read_mode says	forced global
write_mode='local' + restore({mode:'global'})	local values (post-mutation)	forced global (deliberate commit)

4.5. Flow diagram: snapshot lifecycle

Time

Request N:

```

|
|—[1] snap = req.location.snapshot()
|   |   reads root, gzip.level, proxy.readTimeout, ...
|   |   via getter (honours read_mode)

```


5. Complete Examples

5.1. Example 1: Rollback on Error

Mutate the location config and roll back if the mutation causes a problem.

```
loc.handler = async (req) => {
  const snap = req.location.snapshot();

  try {
    const cfg = JSON.parse(await req.readBody());
    req.location.root = cfg.root;

    if (cfg.proxy) {
      req.location.proxy.readTimeout = cfg.proxy.readTimeout;
      req.location.proxy.connectTimeout = cfg.proxy.connectTimeout;
    }

    // Test the new config with a probe request
    const ok = await probeBackend(req);
    if (!ok) {
      throw new Error('probe failed');
    }

    req.json({ applied: true });
  } catch (e) {
    snap.restore();
    req.respond(500, {}, 'Config apply failed: ' + e.message);
  }
};
```

5.2. Example 2: Atomic Two-Phase Apply

Validate locally, then commit globally.

```
loc.handler = async (req) => {
  // Phase 1: validate in isolation
  req.location.setWriteMode('local');
  req.location.setReadMode('local');

  const cfg = JSON.parse(await req.readBody());
  const orig = req.location.snapshot(); // capture current state

  req.location.root = cfg.root;

  const valid = await validateConfig(req.location);
  if (!valid) {
```

```

    orig.restore({ mode: 'local' });    // revert local copy
    req.respond(400, {}, 'invalid config');
    return;
}

// Phase 2: commit validated values globally
const validated = req.location.snapshot(); // capture /local/ values
validated.restore({ mode: 'global' });    // write them to global

req.json({ committed: true });
};

```

5.3. Example 3: Upstream Peer Weight Balancing with Rollback

```

loc.handler = async (req) => {
  const ups = nginx.http.upstreams.find(u => u.name === 'backend');
  const snap = ups.snapshot();           // save all peer weights

  try {
    // Rebalance weights based on health check results
    const health = await checkHealth(ups);
    ups.peers.forEach((p, i) => {
      p.weight = health[i].healthy ? health[i].score : 0;
    });

    // Verify at least one peer is alive
    const alive = ups.peers.some(p => p.weight > 0);
    if (!alive) {
      throw new Error('all peers down after rebalance');
    }

    req.json({ rebalanced: true });
  } catch (e) {
    snap.restore();                     // restore original weights
    req.respond(503, {}, e.message);
  }
};

```

5.4. Example 4: Single-Peer Snapshot for Fine-Grained Rollback

```

loc.handler = async (req) => {
  const ups = nginx.http.upstreams.find(u => u.name === 'backend');
  const peer = ups.peers.find(p => p.address === req.getVar('arg_peer'));

```

```

if (!peer) {
  req.respond(404, {}, 'peer not found');
  return;
}

const snap = peer.snapshot();    // snapshot only this peer

peer.down    = true;             // take peer out of rotation
peer.weight  = 0;

// Wait for in-flight requests to drain
await nginx.setTimeout(5000);

const drained = await verifyDrained(peer.address);
if (!drained) {
  snap.restore();                // put peer back on failure
  req.respond(503, {}, 'drain failed');
  return;
}

req.json({ drained: true, peer: peer.address });
};

```

5.5. Example 5: A/B Configuration Experiment

```

loc.handler = (req) => {
  // Route 10% of requests to experimental config
  const isExperiment = Math.random() < 0.10;

  if (!isExperiment) {
    req.sendfile('/index.html');
    return;
  }

  // Local-only mutation – no global side-effect
  req.location.setWriteMode('local');
  req.location.setReadMode('local');

  req.location.root          = '/var/www/experiment';
  req.location.sendfile      = false;
  req.location.defaultType   = 'text/html; charset=utf-8';

  req.sendfile('/index.html');
  // local snapshot is automatically discarded when the request ends
};

```

5.6. Example 6: Config Snapshot for Audit

```
loc.handler = (req) => {
  // Return a JSON representation of current location config
  const snap = req.location.snapshot();

  // NginxSnapshot is opaque – to read its values, read the location directly
  const config = {
    root: req.location.root,
    sendfile: req.location.sendfile,
    keepaliveTimeout: req.location.keepaliveTimeout,
    proxy: req.location.proxy ? {
      readTimeout: req.location.proxy.readTimeout,
      connectTimeout: req.location.proxy.connectTimeout,
      buffering: req.location.proxy.buffering,
    } : null,
  };

  req.json(config);
};
```

5.7. Example 7: Guarded Global Mutation

Prevent concurrent mutations from interleaving.

```
let mutationInProgress = false;

loc.handler = async (req) => {
  if (mutationInProgress) {
    req.respond(429, {}, 'mutation in progress, retry');
    return;
  }

  mutationInProgress = true;
  const snap = req.location.snapshot();

  try {
    const cfg = JSON.parse(await req.readBody());
    req.location.root = cfg.root;
    req.location.etag = cfg.etag ?? true;
    // ... more fields ...

    req.json({ applied: true });
  } catch (e) {
    snap.restore();
    req.respond(500, {}, e.message);
  } finally {
    mutationInProgress = false;
  }
};
```

```
}  
};
```

6. API Reference

6.1. location.setWriteMode(mode)

Sets the write destination for property assignments on this location and its sub-objects.

Parameter	Description
<code>mode</code>	'global' 'local' 'both'

Also propagates the new mode to `rctx→write_mode` so that sub-object setters (`proxy`, `gzip`, etc.) route writes consistently.

6.2. location.setReadMode(mode)

Sets the read source for property getters on this location.

Parameter	Description
<code>mode</code>	'global' — always read from global shared struct. 'local' — prefer per-request snapshot; fall back to global.

6.3. location.snapshot()

Captures the current readable state of the location and all of its mutable sub-objects.

```
NginxSnapshot location.snapshot()
```

Returns a `NginxSnapshot` object. The snapshot is a point-in-time copy of every settable property value, read through the current `read_mode`.

Null/undefined property values are silently excluded.

6.4. upstream.snapshot()

Captures all peers in the upstream's current `peers[]` array.

```
NginxSnapshot upstream.snapshot()
```

Each peer becomes a separate node in the snapshot. The snapshot is ordered (peer 0 first, backup peers last).



If peers are added or removed between `snapshot()` and `restore()`, the snapshot nodes point to the original peer objects (which remain valid as long as the upstream zone memory is not recycled). Restoring a snapshot after `removePeer()` may write to a peer that is no longer in the active list — this is harmless but produces no observable effect.

6.5. `peer.snapshot()` / `rrPeer.snapshot()`

Captures the five mutable fields of a single peer.

```
NginxSnapshot peer.snapshot()
NginxSnapshot rrPeer.snapshot()
```

6.6. `snap.restore([options])`

Re-applies all captured values through the standard setter path.

```
undefined snap.restore()
undefined snap.restore({ mode: 'global' | 'local' | 'both' })
```

Option	Description
<code>mode</code>	Optional string. When supplied, overrides the current <code>write_mode</code> for the duration of this restore call only. Original mode is restored after all fields are written.

Behaviour:

- Idempotent — calling `restore()` multiple times is safe.
- Best-effort — individual setter errors are silently cleared.
- Mode override applies to the location node and to `rctx>write_mode` so sub-object setters also use the requested mode.
- In config phase (no active request) `restore({ mode: 'local' })` is a no-op for sub-objects because there is no per-request context; location core fields are still written to `loc_op>write_mode`.

6.7. `nginx.suspendAcceptance()`

Removes all listening socket read events from this worker's event loop. New TCP connections queue in the OS socket backlog; other workers continue accepting normally.

```
undefined nginx.suspendAcceptance()
```

Throws `TypeError` if called outside a worker process. Throws `InternalError` if the underlying event operation fails.

6.8. `nginx.resumeAcceptance()`

Re-adds the listening socket read events removed by `suspendAcceptance()`. No-op if acceptance was not suspended.

```
undefined nginx.resumeAcceptance()
```

Throws `TypeError` if called outside a worker process.



For cross-worker suspension, use `nginx.suspendAllWorkers()` / `nginx.resumeAllWorkers()` (documented in the *for all workers* guide).

6.9. `NginxSnapshot` (opaque object)

`NginxSnapshot` is an opaque QuickJS object with class name `"NginxSnapshot"`. Its only JS-visible method is `restore()`. The captured values are not directly accessible from JavaScript.

The snapshot is garbage-collected when the last JS reference to it is dropped; no manual cleanup is required.

7. Design Notes

7.1. Internal snapshot vs JS snapshot

The internal C-level snapshot (`ngx_js_ensure_core_snapshot`, `ngx_js_ensure_module_snapshot`) is an **automatic** mechanism triggered by `write_mode='local'` setters. It operates at the C struct level and is invisible to JavaScript.

The JS-level `snapshot()` / `restore()` operates at the **property** level: it calls the existing getters and setters. The two mechanisms are complementary:

- `write_mode='local'` + internal snapshots → isolation of mutations.
- `snapshot()` + `restore()` → rollback of mutations.

7.2. Why `restore()` is best-effort

Some setters may legitimately fail (e.g. `alias` on a root-based location throws a type error). Because `snapshot()` silently skips null/undefined values (and `alias` returns null for root-based locations), such fields are never captured. The `restore()` "best-effort" guarantee handles any remaining edge cases — e.g. a location type changed between snapshot and restore — without crashing the request.

7.3. Snapshot scope for `upstream.snapshot()`

`upstream.snapshot()` captures the **current** peer array. If peers are added via `addPeer()` after the snapshot, those new peers are not in the snapshot and will not be removed by `restore()`. Similarly, if peers are removed via `removePeer()`, restoring field values to a removed peer has no effect on the live upstream.

For a complete "undo `addPeer/removePeer`", use the `addPeer()/removePeer()` methods directly rather than relying on `snapshot()`.